223 Project (Terminal Trivia Game TCP)

Abbas Ahmadi, 2221190136

Naser Salamat, 2192115811

Ali Oudah, 2211114760

Abbas's Part: Session Management Module and Dynamic Linking

- **Session overview:**

  The module is responsible for controlling the gameplay flow once the required number of players has joined the lobby. This module isolates the game session logic from the networking code in the server and keeps the implementation modular.

  The following function was implemented in server.c in the childe process:

  run_game_session(Player players[], int count, Question questions[], int num_q);

- **session.h**

  The session.h header file defines the interface for the session module. It declares the **run_game_session()** function and includes the required data structure from other modules.

  The header includes:
  - game.h for Player.
  - questions.h for Question
  - csapp.h for robust I/O

  This session header provides modular separation.

- **session.c**

  The session.c file implements the gameplay loop for a single game session. It provides the following steps:

  1. Initializes buffered I/O for each connected player.
  2. Rests player scores at the start of the session.
  3. Iterates through a fixed number of questions.
  4. Sends one question at a time to all players.
  5. Receives one answer from each player.
  6. Calls the game logic module to evaluate answers.
  7. Announces the winner after all questions are completed.

- **Dynamic Linking:**

  Dynamic Linking is implemented using a shared library: **libgame.so**

  And it's built from multiple modules:

  - game.c
  - questions.c
  - session.c
  - protocol.c
  - csapp.c

  In the Makefile:

  gcc -fPIC -shared -o libgame.so <all modules>

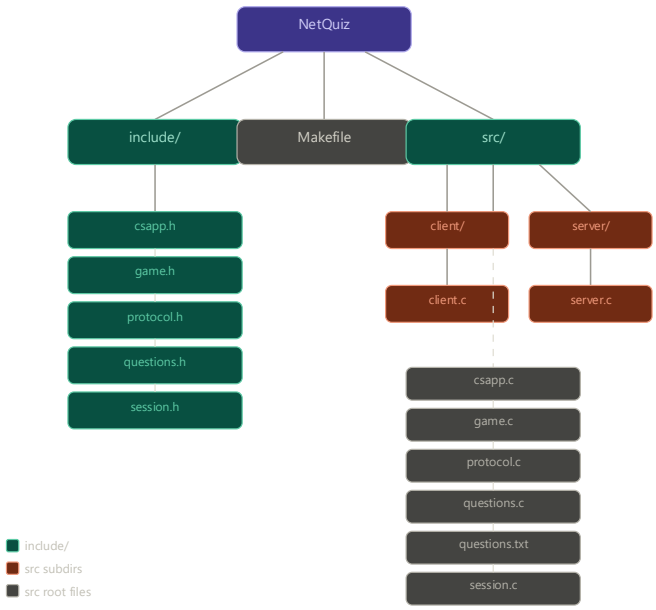
  Then the server and client executables are then dynamically linked against this library using:

  -L. -lgame

Ali's Part: Server, Client, and Build System

- **File Structure**

  The project was organized into a structured directory layout. Of course, it makes our life easier to understand the project and each part**. The following structure was used:**



- **Server**

  The server creates a listening socket on a specified port and waits for players to connect. Once a client connects the server, receives the player nickname and displays a message to the server that the player has joined.

  The following logic was implemented in **server.c:**

- Create a TCP listening socket using **Open_listenfd()** function
- Accept client connections using **Accept()** function
- Receive the nickname sent by the client using **Rio I/O**

Note: All the functions designed/created by csapp.c and of course we deal with file descriptors.

- **Client**
  The client connects to the server using the provided IP[hostname] and port number. Once connected it prompts the player for a nickname and once the player sends it. It will displays a welcome message to the client.

  The following logic was implemented in **client.c:**
  - Connect to the server using TCP using **Open_clientfd()** function
  - Prompt the user for a nickname using **Fgets()** function
  - Send the nickname to the server using **Rio I/O**

  Note: All the functions designed/created by csapp.c and of course we deal with file descriptors.

- **Makefile**
  A Makefile was created to simplify compiling and managing the project. Since we need to compile each separate as client, server, and shared source files. The Makefile supports:
  - Compiling both the server and client
  - Compiling only the server
  - Compiling only the client
  - Removed old executable files [clean only server and client executables]
  - Rebuilding the entire project from scratch [clean all]

Naser's Part: Game Logic and Question Handling Module

questions.c

Question file handling:

I implemented the logic for loading trivia questions from questions.txt and storing them in the proper format so they can be used during the game session.

game.c

Answer checking:

I implemented the logic that checks each player's answer against the correct answer for the current question. This allows the server to determine whether the submitted answer is correct or incorrect.

Score tracking:

I implemented the score update logic during gameplay. Player scores are reset at the start of the session, updated after each question, and then used at the end of the game to determine the winner.

protocol.c：

This file defines how game-related messages are structured and exchanged between the client and server during the session.

questions.txt:

This file contains the quiz data including questions, answer choices, and correct answers.
